Governing Human–AI Collaborative Project Workflows in Multi-Platform Digital Delivery: A Design Science Study of Auditable Handoffs and Idempotent Release Operations

HuaiCheng Sheng[1,*]

[1]Agentic Work Lab, Hangzhou Chengxin Holding Group Co., Ltd., Hangzhou, Zhejiang, China

[*]Corresponding author. E-mail: `ZackSheng@gmail.com` | ORCID: `0009-0003-2900-6377` | GitHub: `https://github.com/whitecarp`

ABSTRACT

Context: As Large Language Model (LLM) agents transition from conversational assistants to autonomous execution actors in digital engineering and media production projects, project management faces a critical governance gap. Multi-agent workflows frequently suffer from uncoordinated role handoffs, non-idempotent external side effects (such as duplicate platform publications following network timeouts), silent quality degradation, and confounded performance attribution.

Objective: Grounded in the Design Science Research (DSR) paradigm and stage-gate project control theory, this study designs, implements, and empirically evaluates an auditable human-in-the-loop (HITL) project governance framework and release operations system (*Agentic Work Lab* and *PublishOps CrossCast*) for multi-agent, multi-platform digital delivery.

Method: We formalize a four-layer governance architecture comprising: (1) a deterministic sole-operator and temporal scheduling gate (`planRun`); (2) cryptographic plan-hash locking (`--confirm CURRENT_PLAN_HASH`) coupled with stream-based SHA-256 media integrity and frame-level quality verification; (3) an idempotent receipt state machine that latches uncertain post-submission outcomes (`UNKNOWN` states) to suppress automatic retry faults across heterogeneous API and browser-automated endpoints; and (4) an independent read-back verification, in-place revision, and HITL exception handoff protocol. The framework is evaluated using longitudinal project artifacts spanning $N = 69$ de-identified operational event logs, $N = 55$ channel publication records, $N = 30$ cryptographically verified video delivery versions (2.21 GB), and 2,737 automated verification test cases across 126 test modules.

Results: Across 69 operational checkpoints involving multi-agent coordination (GPT/Codex, Claude, and DeepSeek under human project management), the governance gates suppressed unauthorized mutations in 68 non-write intervals while executing exactly one authorized write (0 duplicate upload replays, 0 new risk incidents). A verified autonomous heartbeat run demonstrated a 24.992 s trigger delay and 31.175 s execution duration with zero task errors. Cross-platform release evaluation across five platforms (Bilibili, YouTube, X, Douyin, and Instagram) confirmed that latching `UNKNOWN` states during programmatic read-back timeouts prevented duplicate publication on Douyin while preserving audit trails. Phase-level latency decomposition on Instagram (423.236 s end-to-end; 412.217 s publication command) revealed that 92.32% (380.551 s) of command duration resides in post-submission platform confirmation wait, identifying the primary vulnerability window for non-idempotent agent retries.

Conclusions: Effective project management of agentic workflows requires shifting from prompt-level persuasion to deterministic stage-gate invariants, cryptographic artifact locking, and idempotent state-machine recovery. All source code, schemas, and de-identified empirical datasets are publicly archived to establish a reproducible benchmark for agentic project governance.

Keywords: Project Management Governance; Human–AI Collaboration; Multi-Agent Workflows; Design Science Research; Idempotent Release Engineering; Stage-Gate Control; PublishOps

Research Highlights

- Proposes a four-layer Design Science governance framework for human–AI collaborative project delivery.
- Formalizes an idempotent receipt state machine that prevents duplicate side effects under `UNKNOWN` network/UI states.
- Evaluates multi-agent handoffs (GPT/Codex, Claude, DeepSeek) across 69 operational events and 30 SHA-256 verified artifacts.
- Demonstrates across 5 platforms that 92.32% of browser-mediated release latency occurs in post-submission confirmation waits.
- Establishes epistemic audit rules separating descriptive cohort metrics ($N = 55$) from confounded causal claims.

1. Introduction

The rapid maturation of Large Language Models (LLMs) and autonomous tool-using agents has initiated a structural transformation in project management, software engineering, and digital operations (Holzmann et al., 2022; Müller et al., 2024; Wang et al., 2024). Unlike traditional automation scripts that execute fixed, deterministic subroutines within narrow integration boundaries, modern agentic workflows deploy heterogeneous foundation models—such as reasoning-oriented coding agents, interface construction agents, and scheduled monitoring agents—to collaborate on complex, multi-stage project deliverables (Wu et al., 2024; Hong et al., 2024; Qian et al., 2024). In digital media production, software release engineering, and commercial outreach operations, these multi-agent topologies promise substantial reductions in manual coordination overhead and turnaround time (Raisch & Krakowski, 2021; Berente et al., 2021).

However, deploying autonomous or semi-autonomous AI agents into real-world production environments exposes severe project governance vulnerabilities that classical project management methodologies (PMI, 2021; Kerzner, 2022) and conversational multi-agent frameworks were not designed to address. From a systems control and project management perspective, four fundamental failure modes emerge when LLM agents interact with shared repositories and external production platforms:

1. **Multi-Agent Handoff Drift and Role Ambiguity:** When multiple specialized models (e.g., GPT/Codex for script synthesis, programmable video generation, and backend verification; Claude for CLI interface construction and layout adaptation; and DeepSeek for scheduled operational monitoring) operate asynchronously on shared project state, context boundaries frequently degrade. Without formal ownership locks and explicit acceptance gates, an operational agent waking up on a cron schedule may re-execute a task already claimed by another instance, overwrite human-approved revisions, or act upon stale schedule checklists (Whyte et al., 2016; Qian et al., 2024).
2. **Non-Idempotent External Side Effects under Platform Uncertainty:** Real-world release operations across heterogeneous digital platforms (such as Bilibili, YouTube, X, Douyin, and Instagram) rely on a complex mixture of official OAuth REST APIs and stateful browser sessions. Network interruptions, asynchronous platform transcoding queues, CAPTCHA verification challenges, or DOM read-back timeouts frequently leave a submission in an indeterminate (`UNKNOWN`) state (Schneider, 1990; Kleppmann, 2017). Naive agentic retry loops interpret a missing confirmation receipt as a failed upload and automatically re-click submission controls, resulting in duplicate public posts, corrupted playlist ordering, or account rate-limiting penalties.
3. **Opaque Quality Degradation in Generative Deliverables:** One-shot generative media and text pipelines frequently introduce localized technical defects—such as single-frame luminance flashes before transitions, audio-visual synchronization drift, clipped floating-point audio tracks, or mobile overlay cropping—that pass superficial completion checks. Without code-driven editable intermediate representations and deterministic signal verification gates, repairing a localized defect requires regenerating the entire deliverable, destroying previously validated segments.
4. **Epistemic Confounding in Project Performance Evaluation:** In both industry white papers and academic literature, the adoption of AI workflows is frequently accompanied by unverified claims of labour savings or audience growth (Raisch & Krakowski, 2021). When comparing historical human-led project outputs against newly launched AI-assisted series, differences in artifact exposure age (e.g., years of cumulative platform traffic versus days), topic selection, and platform recommendation dynamics severely confound naive metric comparisons. Furthermore, treating generated draft volume as equivalent to verified, production-ready delivery obscures the substantial human review and rework required to achieve release acceptance.

To address these challenges, this study adopts the Design Science Research (DSR) methodology (Hevner et al., 2004; Peffers et al., 2007; Gregor & Hevner, 2013; Venable et al., 2016) to conceptualize, architect, implement, and empirically evaluate a comprehensive human-in-the-loop (HITL) project governance framework and release operations system: **Agentic Work Lab** and its standalone multi-platform release engine **PublishOps CrossCast** (`vpub`). Rather than treating AI governance as an informal prompt-engineering exercise, we formulate multi-agent project delivery as a verifiable stage-gate control system (Cooper, 1990; Bianchi et al., 2022) backed by cryptographic artifact locking, frame-accurate quality gates, and fault-tolerant state-machine transitions.

Specifically, this paper investigates the following three research questions (RQs):

- **RQ1 (Governance Architecture & Handoff Control):** How can stage-gate project management principles be formalized into deterministic software invariants to govern multi-model handoffs (GPT/Codex, Claude, and DeepSeek) and prevent unauthorized or duplicate autonomous actions?
- **RQ2 (Idempotent Release Engineering & Fault Recovery):** How can a persistent receipt state machine coordinate multi-platform digital releases, in-place revisions, and authorized deletions across hybrid API and browser-automated channels when post-submission verification encounters timeouts or human authentication barriers?
- **RQ3 (Empirical Latency, Quality & Epistemic Validation):** What are the empirical operational characteristics, latency bottlenecks, multi-modal defect repair patterns, and methodological boundaries observed when deploying this governance architecture across longitudinal multi-platform production workflows?

The primary contributions of this work are fivefold:

1. **A Four-Layer Agentic Project Governance Framework:** We synthesize stage-gate project control (Cooper, 1990) and human-centered automation theory (Parasuraman et al., 2000; Shneiderman, 2020) into four enforceable governance layers: (i) Sole-Operator and Temporal Scheduling Gate, (ii) Cryptographic Plan-Hash and Multi-Modal Artifact Integrity Gate, (iii) Idempotent Receipt State Machine with `UNKNOWN`-State Suppression, and (iv) Independent Read-Back, In-Place Revision, and HITL Verification Handoff.
2. **Formal Algorithms for State-Locked Release Operations:** We present three formal algorithms governing deterministic job planning (Algorithm 1), idempotent publication with pre-commit receipt latching (Algorithm 2), and identity-bound in-place metadata revision and durable-consent deletion (Algorithm 3).
3. **Open-Source Reference Implementation (*PublishOps CrossCast*):** We detail the engineering architecture and longitudinal evolution (from `v0.2.2` through `v0.16.0` and PR #30, commit `75b7abc`) of *PublishOps CrossCast*, comprising 123 Python source modules (27,095 lines of code) and 126 automated test modules (37,155 lines of test code across 2,737 test functions), alongside zero-dependency Node.js verification scripts in *Agentic Work Lab*.
4. **Longitudinal Empirical Trace and Latency Bottleneck Discovery:** We evaluate the system across 69 de-identified operational event records, 30 SHA-256 verified video delivery versions (2.21 GB), frame-level defect scans across 13,034 frames, and live cross-platform publication across five major platforms (Bilibili, YouTube, X, Douyin, and Instagram). Our phase-by-phase timing decomposition demonstrates that post-submission platform confirmation accounts for 92.32% (380.551 s out of 412.217 s) of browser-mediated publication latency, empirically proving why persistent pre-wait commit latching is indispensable.
5. **Methodological Protocol for Auditable AI Project Reporting:** We establish and demonstrate a strict reporting protocol that separates primary observational receipts from causal inferences, preventing survivorship bias and exposure-age confounding when evaluating human-led (n = 45) versus AI-assisted (n = 10) project cohorts.

2. Related Work and Theoretical Background

2.1 Project Management Governance and Stage-Gate Control in the AI Era

Classical project management theory emphasizes structured control mechanisms—such as Work Breakdown Structures (WBS), configuration management, and Stage-Gate reviews—to manage technical uncertainty and enforce accountability across project phases (Cooper, 1990; Whyte et al., 2016; PMI, 2021; Kerzner, 2022). In Cooper's Stage-Gate model and subsequent hybrid agile–stage-gate frameworks (Bianchi et al., 2022), iterative execution within a development stage is separated from downstream commitment by formal decision gates. Each gate comprises predefined deliverables, explicit quality criteria, and an authoritative gatekeeper who decides whether to proceed, recycle for rework, hold, or terminate the activity.

Recent scholarship in project management has begun examining how artificial intelligence alters project governance, decision-making authority, and human–machine agency (Holzmann et al., 2022; Müller et al., 2024). Raisch & Krakowski

(2021) highlight the fundamental tension between automation and augmentation in management: over-automation (removing human judgment from critical control loops) leads to systemic drift and brittle failure modes under novel edge cases, whereas isolated augmentation fails to capture end-to-end operational scalability. Similarly, foundational work in human factors by Parasuraman et al. (2000) and Shneiderman (2020) demonstrates that high-reliability socio-technical systems must combine high automation capability with high human control through transparent audit trails, explicit pre-authorization, and graceful exception handoffs. Despite these theoretical advances, existing project management literature largely relies on conceptual typologies or expert Delphi surveys (Holzmann et al., 2022); there remains a critical gap in translating stage-gate governance into concrete, software-enforced invariants for live, multi-model AI agents.

2.2 Multi-Agent LLM Workflows and Coordination Vulnerabilities

The emergence of reasoning-and-acting paradigms such as ReAct (Yao et al., 2023) and Reflexion (Shinn et al., 2023) has catalyzed multi-agent collaborative frameworks including AutoGen (Wu et al., 2024), MetaGPT (Hong et al., 2024), and ChatDev (Qian et al., 2024). MetaGPT encodes Standardized Operating Procedures (SOPs) into role-specific prompts (e.g., product manager, architect, engineer, QA), while ChatDev structures software development as a sequential chat chain. While effective on self-contained coding benchmarks (Wang et al., 2024), deploying conversational multi-agent systems into continuous, multi-platform production operations exposes severe limitations:

- **Absence of Cross-Process Execution Locks:** Prompt-based SOPs do not prevent two agent sessions (e.g., an interactive coding session and a background scheduled heartbeat) from concurrently mutating the same local browser profile or checklist file.
- **Lack of Cryptographic Plan-to-Artifact Binding:** In conversational handoffs, an agent may approve a text summary while the underlying binary artifact on disk remains an older, unpatched version or suffers from silent read corruption.
- **Unbounded Retry Loops on External Timeouts:** Standard agent tool-calling loops treat HTTP timeouts or DOM selector failures as transient errors and automatically re-invoke the tool, causing catastrophic duplicate publications when the external platform actually accepted the initial submission.

2.3 DevOps, Release Engineering, and Idempotent State Machines

In software engineering, Continuous Integration and Continuous Delivery (CI/CD) pipelines mitigate deployment risk through automated build verification, immutable artifact digests, and staged rollouts (Humble & Farley, 2010; Bass et al., 2015; Adams & McIntosh, 2016; Shahin et al., 2017; Leite et al., 2019). A cornerstone of fault-tolerant distributed systems and transaction processing is *idempotency*—the guarantee that repeating an operation or recovering from an interrupted request does not produce duplicate side effects (Schneider, 1990; Gray & Reuter, 1992; Kleppmann, 2017). Table 1 contrasts existing multi-agent frameworks, CI/CD pipelines, and open-source upload scripts against the proposed *Agentic Work Lab* and *PublishOps CrossCast* architecture across eight governance dimensions.

Table 1: Comparative analysis of multi-agent frameworks, release pipelines, and the proposed governance architecture across eight socio-technical control dimensions.

Framework / System	Multi-Model Handoffs	Temporal & Owner Gate	Crypto Plan & SHA-256 Lock	Hybrid API + Pipe Browser	`UNKNOWN` Latch (Zero Retry)	Identity-Bound Edit/Delete	Epistemic & Non-Causal Audit
AutoGen (Wu et al., 2024)	Conversational chat loops	No native lock	No	External tools only	No (standard retry)	No	No
MetaGPT (Hong et al., 2024)	Prompt-level SOP roles	Sequential memory	No	No	No	No	No
ChatDev (Qian et al., 2024)	Dual-agent chat chain	Phase-level only	Git commit only	No	No	No	No
Standard CI/CD (Humble & Farley, 2010)	Static pipeline stages	Runner queue locks	Artifact SHA-256	REST APIs only	API-dependent	Rollback scripts	Build logs only
Open-Source Upload Helpers (e.g., SAU)	Single-script execution	Cron timer only	No plan-hash lock	Browser form fill	No (risks double post)	No read-back verification	No
Agentic Work Lab & CrossCast (Ours)	**File-mediated state handoffs (GPT/Claude/DS)**	**Deterministic `planRun` + OS lock**	**`--confirm` Plan Hash + Stream SHA-256**	**OAuth API + Private Pipe Browser**	**Durable `f_submit` latch + `UNKNOWN` hold**	**`--verify-only` + Absence Read-Back**	**Enforced machine-readable limits**

3. Research Methodology

3.1 Design Science Research (DSR) Framework

We structure this investigation following the six-step Design Science Research Methodology (DSRM) established by Peffers et al. (2007) and the evaluation guidelines of Hevner et al. (2004), Gregor & Hevner (2013), and Venable et al. (2016):

1. **Problem Identification and Motivation:** Uncontrolled multi-agent handoffs, duplicate external writes during uncertain platform states, localized defects in generative media, and confounded efficiency claims in AI-assisted project delivery (Section 1).
2. **Definition of Solution Objectives:** Establish a deterministic governance architecture that enforces sole-operator scheduling, cryptographic plan-to-artifact binding, zero-duplicate submission under `UNKNOWN` states, surgical frame/audio defect repair, and strict separation of observational evidence from causal inference.
3. **Design and Development:** Formulation of the four-layer governance model, mathematical invariants, and formal algorithms (Section 4), realized through two coupled open-source artifacts: *Agentic Work Lab* and *PublishOps CrossCast* (Section 5).
4. **Longitudinal Demonstration:** Deployment of the artifacts across an authentic multi-series digital media production and cross-platform publishing laboratory spanning two episodic series (*GPT Duty Log* Ep 01–06 and *Whale Girl Shift Log* Ep 01–07), standalone experimental studies, and the cross-platform short film *I Watched It For You (v3)*.
5. **Multi-Method Empirical Evaluation:** Following the Framework for Evaluation in Design Science (FEDS; Venable et al., 2016), we combine naturalistic trace evaluation (N = 69 operational events, 5-platform live release, and N = 55 cohort records) with rigorous offline fault-injection verification (2,737 unit/integration tests in *CrossCast* and 10 workflow gate tests in *Agentic Work Lab*) (Section 6).
6. **Archival Communication:** Publication of all source code, SHA-256 verified media deliverables (N = 30 versions, 2.21 GB), and machine-readable JSON/CSV receipts to enable independent reproduction.

3.2 Epistemic Hygiene and Evidence Classification Protocol

A central methodological contribution of this study is the enforcement of an explicit epistemic hygiene protocol across all project documentation and data exports (Runeson & Höst, 2009). For every research topic and operational update, the

laboratory records four distinct elements: **(i) the original claim, (ii) visible primary evidence, (iii) an independent verification test, and (iv) applicable methodological limits**. Specifically:

- **Separation of Observation and Causal Inference:** Direct platform observations (e.g., lifetime views, 24-hour completion rates, or command durations) are stored with exact sampling timestamps and provenance notes. Causal inferences (e.g., asserting that an AI editing technique increased algorithmic recommendations) are rejected unless backed by controlled, age- and topic-matched experiments.
- **Granular State Differentiation:** The system distinguishes between local preparation, dry-run rehearsal, sign-in preflight, submission execution, platform review queues, and verified public visibility. UI observations and API verifications occupy separate state fields; an unverified or timed-out read-back is recorded as `UNKNOWN` rather than coerced into success or failure.
- **Immutable Snapshot Preservation:** Repeated data collections are archived as separate observations (e.g., retaining the 2026-10-28 afternoon 52-record snapshot separately from the evening 55-record snapshot and subsequent daily updates from 29 October to 3 November) rather than overwriting historical files.
- **Explicit Tagging of Unexecuted Plans:** Pre-registered experimental designs (such as the blinded identical-task multi-model merchant evaluation) and unmeasured optimizations (such as live concurrent-publication wall-clock speedup) are explicitly marked as unrun plans until live empirical receipts are generated.

4. The Four-Layer Human–AI Project Governance Architecture

To answer **RQ1** and **RQ2**, we formalize the project governance architecture as a four-layer hierarchical control system mediating between the human Project Manager, specialized LLM agents, local media/code artifacts, and external publishing platforms (Figure 1).

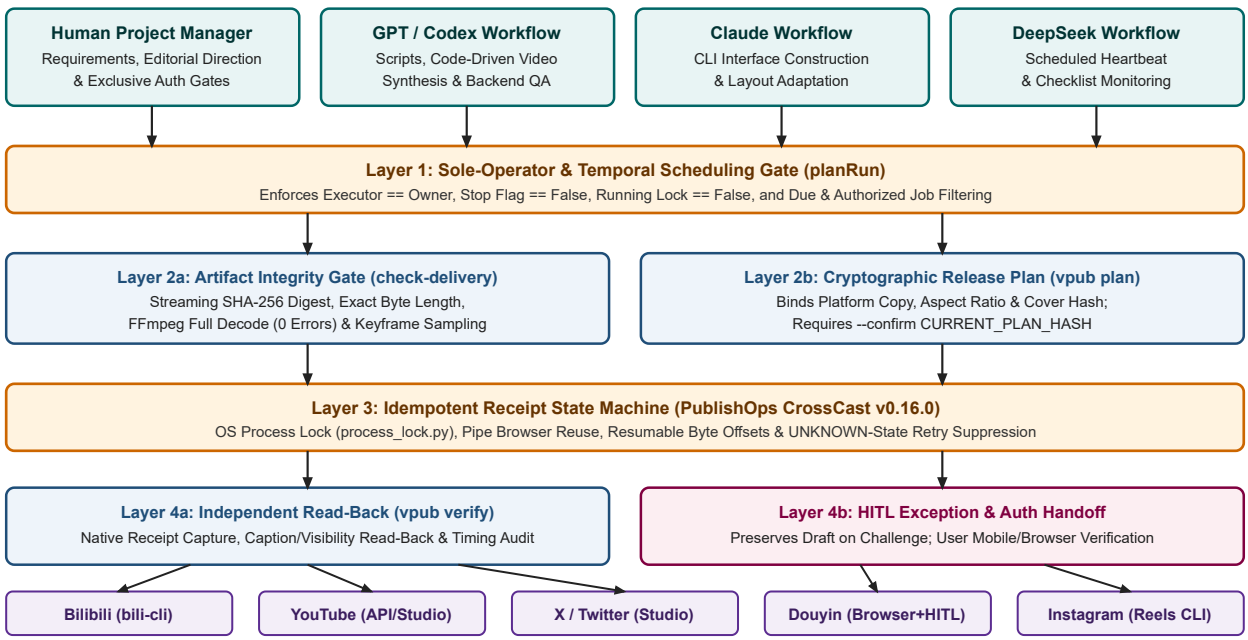


Figure 1. The four-layer human–AI collaborative project governance and multi-platform release architecture implemented in *Agentic Work Lab* and *PublishOps CrossCast*.

4.1 Multi-Agent Role Allocation and Handoff Boundaries

Within the laboratory environment, project execution is partitioned across four distinct actors: the Human Project Manager and three specialized AI agent workflows (GPT/Codex, Claude, and DeepSeek). Rather than allowing agents to communicate via unstructured conversational loops, all inter-agent handoffs are mediated through version-controlled state files

(`operations-checklist.md`, `delivery-manifest.json`, and `production-update-*.json`). Table 2 defines the task scope, handoff artifacts, and operational constraints for each actor.

Table 2: Multi-agent role allocation, handoff artifacts, and governance boundaries across the project lifecycle.

Project Actor	Assigned Functional Scope	Observable Evidence & Handoff Artifacts	Enforced Governance & Safety Constraints
Human Project Manager (Sole Operator / Architect)	Requirements definition, editorial direction, creative acceptance, platform authentication, and final release authorization.	Explicit authorization events (`USER_AUTHORIZED_*`), mobile/browser verification receipts, and plan confirmation hashes.	Retains exclusive sign-off on public releases, deletions, and schedule overrides; resolves `UNKNOWN` platform states.
GPT / Codex Workflow (Production & Backend QA)	Topic/merchant research, script synthesis, programmable video timeline assembly, FFmpeg encoding checks, and backend repository review.	SHA-256 verified `.mp4` exports, frame-sampling logs, decode error counts, and pull request verification records.	Cannot self-authorize public platform writes; unapproved or in-progress renders (`RENDER_OR_QA_PENDING`) are excluded from release.
Claude Workflow (Interface & Tool Construction)	CLI interface construction, layout adaptation, portrait cut tooling, and developer user-experience refinements in *CrossCast*.	Scoped pull requests (e.g., PR 8 capability list, PR 9 first-run checks, PR 10–30 workflows) and unit test suites.	Changes remain commit-scoped until verified by regression suites; local mock tests are never reported as live speedups.
DeepSeek Workflow (Scheduled Operations Monitor)	Hourly automated heartbeat inspections, reading operational checklists, checking due times, and synchronizing user instructions.	Machine-readable execution traces (`scheduled-run-evidence.json`) and de-identified event logs (`operations-events.json`).	Strictly read-only when no authorized task is due; forbidden from launching duplicate runs or retrying uncertain submissions.

4.2 Layer 1: Deterministic Sole-Operator and Temporal Scheduling Gate

To prevent race conditions and unauthorized autonomous execution during scheduled heartbeats, Layer 1 enforces a pure offline decision function G_{plan} before any network or platform adapter is invoked. Let $S = (e, o, f_{\text{stop}}, f_{\text{run}}, t_{\text{now}}, J)$ denote the operational context tuple, where e is the requesting executor identifier, o is the registered instance owner, $f_{\text{stop}} \in \{\text{true}, \text{false}\}$ is the emergency halt flag, $f_{\text{run}} \in \{\text{true}, \text{false}\}$ indicates whether an execution lock is active, $t_{\text{now}} \in \mathbb{R}^{+}$ is the current epoch timestamp, and $J = \{j_1, \ldots, j_m\}$ is the set of candidate jobs.

$$G_{\text{plan}}(S) = [\ (\texttt{skip}, \texttt{not_owner}, \emptyset) \text{ if } e = \emptyset \vee e \neq o;\quad (\texttt{skip}, \texttt{stop_flag}, \emptyset) \text{ if } f_{\text{stop}} = \text{true};\quad (\texttt{skip}, \texttt{already_running}, \emptyset) \text{ if } f_{\text{run}} = \text{true};\quad (\texttt{plan_only}, \texttt{checked}, D) \text{ if } D \neq \emptyset;\quad (\texttt{no_work_due}, \texttt{checked}, \emptyset) \text{ otherwise }] \tag{1}$$

where the due set D is strictly filtered as:

$$D = \{\, id_k \mid j_k \in J \wedge done_k = \text{false} \wedge auth_k = \text{true} \wedge t_{\text{due},k} \leq t_{\text{now}} \,\} \tag{2}$$

Furthermore, for any automated run record $R = (\text{trig}, \text{status}, f_{\text{manual}}, t_{\text{start}}, t_{\text{end}}, E_{\text{err}})$, post-execution verification $V_{\text{sched}}(R)$ holds if and only if:

$$V_{\text{sched}}(R) \Leftrightarrow (\text{trig} = \texttt{automation_heartbeat_scheduled}) \wedge (\text{status} = \texttt{VERIFIED_AUTOMATIC_RUN_COMPLETED}) \wedge (f_{\text{manual}} = \text{false}) \wedge (0 < t_{\text{start}} \leq t_{\text{end}} < \infty) \wedge (|E_{\text{err}}| = 0) \tag{3}$$

Algorithm 1: Deterministic Sole-Operator & Temporal Scheduling Gate (`planRun` & `verifyScheduledRun`)

Input: Context tuple $S = (e, o, f_{stop}, f_{run}, t_{now}, J)$; Run trace record R
Output: Gate decision tuple (decision, reason, D) and boolean verification result v

1: $t_{cur} \leftarrow$ ParseEpoch(t_{now})
2: **if** ¬IsFinite(t_{cur}) **then throw** Error("Invalid current time")
3: **if** $e = \emptyset \lor e \neq o$ **then return** ("skip", "not_owner", $\emptyset$)
4: **if** f_{stop} = true **then return** ("skip", "stop_flag", $\emptyset$)
5: **if** f_{run} = true **then return** ("skip", "already_running", $\emptyset$)
6: $I_{seen} \leftarrow \emptyset$; $D \leftarrow$ []
7: **for each** job $j_k = (\text{id}_k, t_{due,k}, \text{auth}_k, \text{done}_k) \in J$ **do**
8: **if** $\text{id}_k = \emptyset \lor \text{id}_k \in I_{seen}$ **then throw** Error("Missing or duplicate job id")
9: $I_{seen} \leftarrow I_{seen} \cup \{\text{id}_k\}$
10: **if** done_k = true $\lor$ $\text{auth}_k \neq$ true **then continue**
11: $t_{at} \leftarrow$ ParseEpoch($t_{due,k}$)
12: **if** ¬IsFinite(t_{at}) **then throw** Error("Invalid due time for " + id_k)
13: **if** $t_{at} \leq t_{cur}$ **then** Append(D, id_k)
14: **end for**
15: decision ← ($|D| > 0$) ? "plan_only" : "no_work_due"
16: **return** (decision, "checked", D)

As formalized in Algorithm 1, `planRun` operates as a strict, side-effect-free admission filter. First, it validates temporal integrity by parsing the reference clock t_{now} into a finite epoch value (lines 1–2). Next, it enforces three ordered circuit-breaker checks: executor ownership verification (line 3), emergency stop-flag inspection (line 4), and active-execution lock detection (line 5). Only when all three preconditions pass does the algorithm iterate over the candidate job set J, maintaining a seen-identifier set I_{seen} to immediately reject malformed or duplicate job identifiers (lines 6–9). Tasks that are already completed (`done` = true) or lack explicit human pre-authorization (`auth` ≠ true) are deterministically bypassed (line 10). Crucially, even when eligible due tasks are identified ($|D| > 0$), Layer 1 returns `plan_only` rather than directly mutating external platforms, requiring downstream layers to validate cryptographic artifact digests and confirmation hashes before execution.

4.3 Layer 2: Cryptographic Plan-Hash Locking and Multi-Modal Artifact Verification

In multi-agent workflows, a subtle but critical vulnerability arises between plan approval and execution: an agent or background process may modify a video export, swap a cover image, or alter platform copy after the human project manager has reviewed the preview. Furthermore, large binary media deliverables (ranging from 2.5 MB to 285.8 MB in our archive) can suffer from incomplete writes, silent disk read corruption, or container-level encoding defects. Layer 2 eliminates this time-of-check to time-of-use (TOCTOU) gap by enforcing two formal invariants prior to release staging:

Governance Invariant 1 (Binary & Multi-Modal Media Integrity): Let $M = \{m_1, \ldots, m_K\}$ be the release manifest where each entry $m_i = (\text{fname}_i, B_i, H_i)$ specifies a basename-sanitized file, exact byte length B_i, and expected SHA-256 digest H_i. An artifact f_i passes verification iff $\text{basename}(\text{fname}_i) = \text{fname}_i \land \text{size}(f_i) = B_i \land \text{SHA256}(f_i) = H_i \land \text{DecodeErrors}(f_i) = 0$.

Governance Invariant 2 (Plan-Hash Confirmation Binding): Before invoking any platform write via `vpub publish`, a canonical release plan P—encapsulating target platforms, account identifiers, video SHA-256 digest, cover SHA-256 digest, aspect ratio, visibility mode, and platform-specific captions—is serialized and hashed as $H(P) = \text{SHA256}(\text{CanonicalJSON}(P))$. Execution is rejected unless the invoking command supplies an explicit confirmation token `--confirm` H_{user} such that $H_{\text{user}} = H(P)$.

To avoid memory exhaustion when verifying multi-hundred-megabyte video deliverables, both `check-delivery.mjs` (in Node.js) and `vpub` (in Python) compute SHA-256 digests using chunked read streams rather than buffering entire files into

memory, and sanitize all manifest file paths via `path.basename` to prevent directory traversal. Beyond cryptographic integrity, Layer 2 enforces domain-specific multi-modal quality gates: every video candidate must pass a full FFmpeg stream decode with zero errors, loudness normalization checks (targeting −15.97 to −17.5 LUFS with zero clipped samples), and automated low-resolution (320×180) frame-by-frame luminance and difference scans to detect single-frame ABA flashes, black-frame dropouts, or shot leakage across edit cuts.

4.4 Layer 3: Idempotent Receipt State Machine and UNKNOWN-State Suppression

To answer **RQ2**, we model cross-platform publication in *PublishOps CrossCast* as a persistent, six-state finite state machine backed by cross-process OS locks (`process_lock.py`) and durable local receipt journals (Figure 2 and Algorithm 2). Let Σ = {`PLANNED`, `PREFLIGHT_OK`, `MEDIA_STAGED`, `SUBMITTED_WAIT`, `UNKNOWN_LATCHED`, `SUBMISSION_VERIFIED`} denote the state space for a target platform p. Each publication attempt maintains an atomic receipt tuple R_p = (key, acct_p, H_{media}, H_{meta}, f_{submit}, σ_p, wid_p, τ_{log}), where $f_{\text{submit}} \in$ {true, false} is the persistent final-commit latch.

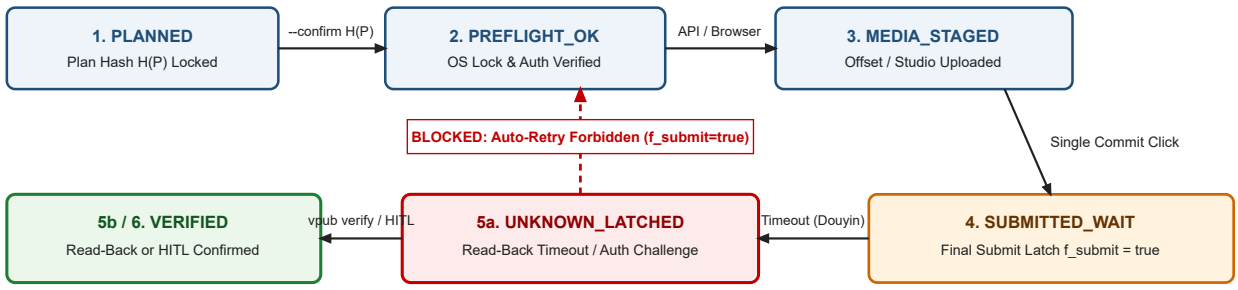


Figure 2. The idempotent receipt state machine in *PublishOps CrossCast*. Once the final-submit latch (`f_submit = true`) is persisted at State 4, any subsequent timeout transitions to `UNKNOWN_LATCHED` (State 5a), deterministically blocking automatic retries.

Algorithm 2: Idempotent Cross-Platform Publication & `UNKNOWN`-State Latching (`vpub publish` & `vpub verify`)

Input: Release plan P, confirmation token H_{user}, target platform p, receipt journal R_p
Output: Updated persistent receipt R_p = (key, acct_p, H_{media}, H_{meta}, f_{submit}, σ_p, wid_p, τ_{log})

if $H_{\text{user}} \neq$ SHA256(CanonicalJSON(P)) **then abort** `PLAN_HASH_MISMATCH`
AcquireProcessLock(p, acct_p) // Recovers stale lock only if OS PID is confirmed terminated
if $R_p.f_{\text{submit}}$ = true **then**
    return ReconcileExistingReceipt(R_p) // Strictly blocks duplicate submission if already latched
end if
VerifyMediaStreamSHA256(P.media_file, P.expected_bytes, $P.H_{\text{media}}$)
session ← AttachPipeBrowserOrOAuthSession(p, acct_p)
StageMediaWithOffsetRecovery(session, P.media_file, P.cover_file, P.caption)
$R_p.f_{\text{submit}}$ ← true; $R_p.\sigma_p$ ← `SUBMITTED_WAIT`; PersistAtomically(R_p)
DispatchSingleFinalCommit(session) // Single Share / Post / Save click
outcome ← AwaitPlatformConfirmationAndReadBack(session, P.expected_identity)
if outcome.verified = true **then**
    $R_p.\sigma_p$ ← `SUBMISSION_VERIFIED`; $R_p.\text{wid}_p$ ← outcome.work_id
else
    $R_p.\sigma_p$ ← `UNKNOWN_LATCHED` // Never re-clicks submit; awaits vpub verify or HITL inspection
end if
PersistAtomically(R_p); ReleaseProcessLock(p, acct_p); **return** R_p

The core fault-tolerance property of Algorithm 2 lies in the strict ordering between receipt latching (line 9) and external commit dispatch (line 10). During preflight and media byte staging (States 1–3, lines 6–8), no public post exists on the target platform; if a network drop occurs during chunked upload, `StageMediaWithOffsetRecovery` safely resumes byte transfer from the last acknowledged offset or re-attaches to the existing studio draft. However, immediately before clicking the final platform commit button (e.g., Instagram *Share*, YouTube Studio *Save*, or Douyin *Publish*), the state machine sets $R_p.f_{\text{submit}} \leftarrow$ true and flushes the receipt atomically to disk (line 9). If the subsequent confirmation wait or read-back verification (line 11) times out or encounters a platform authentication challenge, the receipt transitions to `UNKNOWN_LATCHED` (line 15). Any subsequent invocation of `vpub publish` detects $R_p.f_{\text{submit}}$ = true at line 3 and routes execution to `ReconcileExistingReceipt` (`vpub verify`) rather than re-submitting the media file, thereby guaranteeing zero duplicate publications under arbitrary post-commit network or UI faults.

4.5 Layer 4: Identity-Bound In-Place Revision and Durable-Consent Deletion

Beyond initial publication, day-to-day channel operations require revising captions, updating covers, assigning chapters, and occasionally deleting test works. Naive automation agents often implement "edit" by deleting and re-uploading a post, or execute deletions without verifying that the target item matches the intended work identity. Layer 4 enforces a strict identity-bound revision and deletion protocol (Algorithm 3), implemented across PRs #20, #21, #25, #29, and #30.

Algorithm 3: Identity-Bound In-Place Revision & Durable-Consent Single-Work Deletion (`vpub edit` & `vpub delete`)

Input: Receipt key K, patch specification ΔP (or delete consent token C_{del}), mode $\in$ {`VERIFY_ONLY`, `APPLY_EDIT`, `DELETE_WORK`}
Output: Reconciled receipt state σ_p and audit log entry

1: $R_p \leftarrow$ LoadReceipt(K); VerifyAccountOwnership(session, R_p.acct$_p$, ΔP.expected_identity)
2: **if** mode $\in$ {`VERIFY_ONLY`, `APPLY_EDIT`} **then**
3: live_meta $\leftarrow$ ReadEditorNativeMetadata(session, R_p.wid$_p$) // Ignores page translations & comments
4: **if** live_meta == ΔP.target_meta **then return** `NO_SAVE_REQUIRED_MATCHED`
5: **if** mode = `VERIFY_ONLY` **then return** `EDIT_MISMATCH_DETECTED`
6: **if** p = `X` $\wedge$ ¬ExposesNativeEditControl(session, R_p.wid$_p$) **then**
7: **abort** `MISSING_EDIT_ACCESS_NO_DELETE_OR_REPOST` // Never deletes or reposts as fallback
8: **end if**
9: ApplyInPlaceMutation(session, R_p.wid$_p$, ΔP) // Preserves video bytes and visibility
10: ReopenEditorAndVerifyReadBack(session, R_p.wid$_p$, ΔP.target_meta)
11: **else if** mode = `DELETE_WORK` **then**
12: VerifyDurableConsentToken(C_{del}, R_p.wid$_p$, $R_p.H_{\text{media}}$)
13: ExecuteSingleWorkDeletion(session, R_p.wid$_p$)
14: RefreshCatalogAndConfirmAbsence(session, R_p.wid$_p$) // Verifies work is absent & neighbors intact
15: **end if**

Algorithm 3 introduces three critical safety guarantees for post-publication lifecycle management. First, before reading or mutating any remote item, `VerifyAccountOwnership` (line 1) validates that the active browser or API session matches `expected_identity`, preventing cross-account contamination. Second, in `VERIFY_ONLY` or `APPLY_EDIT` mode, the adapter inspects the native editor fields directly (ignoring browser-injected page translations or user comment text, line 3); if a platform account lacks native edit capability (such as a standard X post without edit access, lines 6–8), the algorithm aborts with `MISSING_EDIT_ACCESS_NO_DELETE_OR_REPOST` rather than destructively deleting and re-uploading the post. Third, when executing an authorized single-work deletion (`DELETE_WORK`, lines 11–14), the system requires a durable consent token bound to the specific work ID and media hash, waits for deletion control readiness (PR #30), and refreshes the creator catalog to verify both that the target work is absent and that neighboring works remain intact (PR #29).

5. System Implementation: Agentic Work Lab and PublishOps CrossCast

5.1 Programmable Video Synthesis and Multi-Modal Quality Gates

Within *Agentic Work Lab*, video production is engineered as an editable, code-driven workflow (`editable-video-workflow.md`) rather than opaque one-shot video generation. Audio narration timelines, character pose transitions, camera movements, and bilingual subtitles are defined programmatically so that localized defects can be repaired surgically without regenerating unaffected segments (Figure 3).

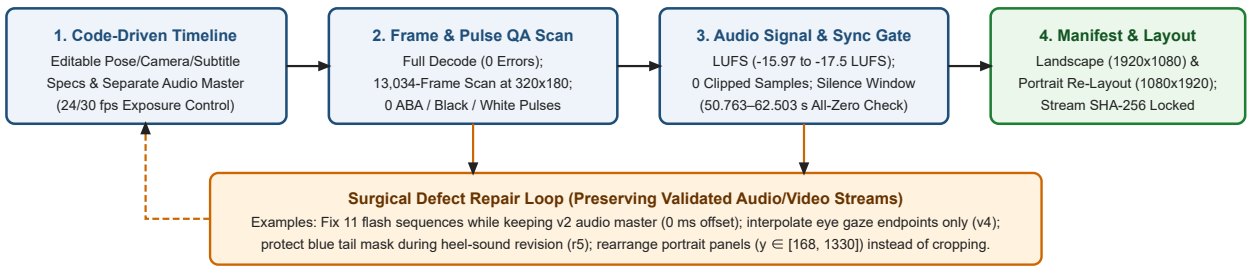


Figure 3. The programmable video synthesis and surgical multi-modal quality verification pipeline in *Agentic Work Lab*.

5.2 CrossCast Two-Slot Concurrent Browser Architecture and Session Reuse

PublishOps CrossCast (public release `v0.16.0`, commit `a46bd19`; main-branch evolution through commit `75b7abc`, PR #30) is implemented in Python across 123 source modules (27,095 LOC) and validated by 126 test modules (37,155 LOC, 2,737 test functions). To overcome the 92.32% post-submission confirmation wait bottleneck without risking browser profile corruption, PRs #11 and #15 (commits `066c49f` and `6f7b62b`) introduced a bounded two-slot concurrent browser scheduler (Figure 4). Each platform retains an isolated account lock and pipe session, allowing up to two browser-automated platforms to execute concurrently and immediately dispatching the next queued platform as soon as a slot completes read-back verification.

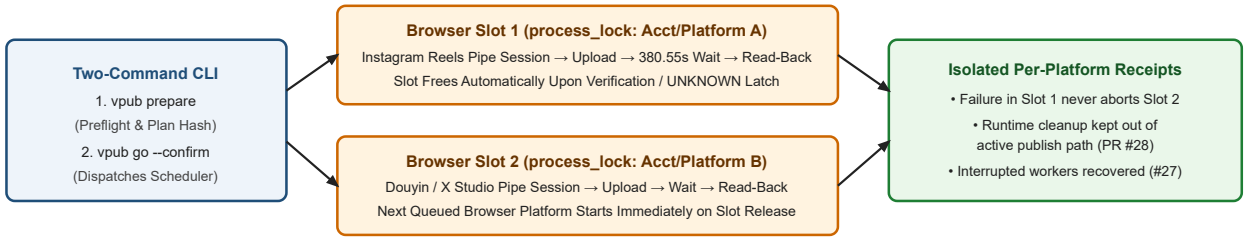


Figure 4. Two-command release workflow (`prepare` / `go`) and bounded two-slot concurrent browser publication architecture in *PublishOps CrossCast* (PRs #10, #11, #15, #27, #28).

6. Empirical Evaluation and Results

6.1 Operational Event Trace Analysis and Write Suppression (RQ1)

Table 3 summarizes the taxonomy and governance metrics across the complete de-identified operational event journal (`data/operations-events.json`, $N = 69$ events) alongside the verified autonomous heartbeat trace (`data/scheduled-run-evidence.json`).

Table 3: Taxonomy and governance metrics across *N* = 69 de-identified operational event records in *Agentic Work Lab*.

Operational Event Category	Events	Share	Reads	Writes	Primary Governance Mechanism Verified
Scheduled Zero-Write Heartbeat & Idle Checks	32	46.38%	9	0	Layer 1 (`planRun`): Suppresses writes when no task is due (`no_work_due`).
Human Governance, Topic & Release Auth Gates	16	23.19%	4	0	Layer 1 & 4: Explicit human sign-off (`USER_AUTHORIZED_*`) & preview reply gates.
Production Delivery, Temporal QA & Revisions	16	23.19%	4	0	Layer 2: Frame/decode checks, visual revision loops & spare preservation.
Station Sampling & Trend Research Queries	4	5.80%	0 (26 CLI)	0	Bounded read-only CLI sampling (`cli_calls = 26`) without platform writes.
Authorized Platform Write Execution	1	1.45%	4	1	Controlled single-write execution with read verification (`new_risk = false`).
Total / Aggregate Audit Metrics	**69**	**100.00%**	**17**	**1**	**Duplicate Replays: 0 | New Risk Events: 0**

Out of 69 operational events, 68 (98.55%) executed with zero platform writes, zero duplicate upload replays, and zero new risk incidents. In the audited DeepSeek scheduled run (`2026-10-28T12:37:13.943+08:00`), the heartbeat triggered after a scheduling delay of 24,992 ms (24.992 s) and completed its four-step inspection in 31,175 ms (31.175 s) with zero task errors and no duplicate trigger.

6.2 Software Verification and Longitudinal Engineering Evolution (RQ1 & RQ2)

Table 4 presents the subsystem code volume and automated fault-injection verification coverage across *PublishOps CrossCast* and *Agentic Work Lab*, while Table 5 documents the longitudinal Design Science evolution across successive commits and pull requests.

Table 4: Subsystem code volume and automated fault-injection verification coverage across *PublishOps CrossCast* and *Agentic Work Lab*.

Subsystem / Verification Category	Files	Test LOC	Test Cases	Key Fault Modes & Invariants Tested
A. PublishOps CrossCast (v0.16.0 / main 75b7abc: 123 source files, 27,095 source LOC)				
1. Workflow, CLI, Scheduling & Notifications	66	17,020	1,266	Two-command `prepare` / `go` gates, plan-hash validation, two-slot concurrency, Gmail self-send claims.
2. Platform Adapters & Publishing (Bili, YT, X, DY, IG)	18	7,016	562	Aspect-ratio guards, FFmpeg cover lead-in synthesis, X post length checks, YouTube Studio zones.
3. Deletions, Edits & Read-Back Verification	24	8,575	485	`expected_identity` binding, `EDIT_MISMATCH` recovery, durable-consent single-work deletion & absence read-back.
4. Collections, Playlists & Catalog Discovery	10	3,577	322	Account-bound collection IDs, partial-page scroll guards, lost-response recovery, playlist deduplication.
5. Idempotency, Receipts, Locks & Recovery	8	967	102	OS process lock recovery after confirmed exit, resumable upload byte offsets, `UNKNOWN` retry blocking.
Subtotal (PublishOps CrossCast)	**126**	**37,155**	**2,737**	**Test-to-Source LOC Ratio: 1.37 : 1**
B. Agentic Work Lab Reproducible Entry Points (Node.js v26.11.1)				
6. Workflow Gate & Merchant Demo (`workflows.test.mjs`)	1	38	10	Non-owner rejection, stop-flag blocking, running-task lock, invalid due-time exception (10/10 passed in 108.5 ms).
7. Archive Cryptographic Integrity (`manifest.json`)	1	—	30	Stream SHA-256 & exact byte checks across 30 downloadable video versions (2,211,716,129 bytes; 0 mismatches).

Table 5: Longitudinal Design Science evolution of *PublishOps CrossCast* from early prototypes to commit `75b7abc` (PR #30).

Version / Commit	Architectural Milestone	Empirical & CI Verification Evidence	Identified Boundary & Subsequent DSR Refinement
`v0.2.2–v0.3.0` (`596e951`)	Initial 5-platform adapters, X Media Studio 16:9 thumbnail persistence, FFmpeg 79.5s cover-lead synthesis.	86 offline tests passed; verified YouTube Studio category change & 1,327-char Instagram caption read-back.	Instagram profile grid cropped landscape cover text; TikTok suspended after HK phone login block.
`v0.15.1` (Lab Snapshot)	OS process lock (`process_lock.py`), private pipe browser context reuse, resumable byte-offset recovery.	417 offline tests passed; atomic API pagination and Douyin/Instagram collection membership controls.	Needed independent CI matrix verification and stricter schedule/pin counterexample checks.
Candidates `eaa1aab` / `80e166b` / `2d54c8f`	Separated development from independent validation; tested X schedule submission and cleanup.	GitHub CI passed on Python 3.10–3.12; 143 + 16 targeted local checks passed; X temp media deleted & confirmed absent.	Counterexample found imprecise pinned-comment ID (live pin held); X schedule row lacked post/media ID pair (kept `SUBMITTED_UNVERIFIED`).
`v0.16.0` Release (`a46bd19` / PR #8–#9)	Public standalone CLI release; shortened default capability list (`--full`) and separated rehearsal from live readiness.	Live 5-platform release of *I Watched It For You v3*; instrumented 412.217 s Instagram timing breakdown.	Post-submit wait consumed 92.32% of publish time; motivated explicit wait progress and bounded concurrency.
PRs #10–#28 (`d97835f` – `439c038`)	Two-command `prepare` / `go` flow (#10), two-slot browser concurrency (#11/#15), YouTube single-Save guard (#16), X edit consistency (#21).	Python 3.13/3.14 support (#14); prompt login return & interrupted worker recovery (#27); runtime cleanup isolated (#28).	Required durable consent and post-deletion absence read-back for single-work deletions across all platforms.
PRs #29–#30 (`3b56bdc` / `75b7abc`)	Durable-consent single-work deletion with absence read-back (#29); deletion control readiness & layout guards (#30).	Full regression suite expanded to **126 test files (37,155 LOC, 2,737 test functions)**; all assertions passing.	Live multi-platform concurrent speedup benchmarking scheduled as next empirical evaluation phase.

6.3 Cross-Platform Release Verification and Latency Bottleneck Analysis (RQ2 & RQ3)

To evaluate the idempotent release architecture under real-world network and platform conditions, we examine the cross-platform release of the 176.4 s short film *I Watched It For You (v3)* (`i-watched-it-for-you-v3-20261102.mp4`, 285,855,005 bytes, SHA-256 `7d6fd76090f7a90b79d8e5af7a52a8a7f5f0c1881fd227556ba0db1c2f92db22`) across five major platforms on 2–3 November 2026 (Table 6), together with the instrumented phase-level latency profile captured on Instagram (Table 7 and Figure 5).

Table 6: Cross-platform publication verification matrix for *I Watched It For You (v3)* (176.4 s, 285,855,005 bytes, SHA-256 `7d6fd760...`).

Platform	Transport Route	Native Receipt State	Verification State	Observed Governance Outcome
Bilibili	`bili-cli` Session Reuse	`REPORTED_PUBLIC`	`PUBLIC_PER_SESSION` (`BV15B4F6hEYr`)	Public availability recorded at 2 Nov 21:00 UTC+8; state preserved without redundant re-query.
YouTube	Official API / Studio Draft Continuation	`SUBMISSION_VERIFIED`	`PUBLIC_VERIFIED` (`AzW6vr8nvDM`)	Continued from original draft; native receipt, bilingual copy, and visibility passed automated read-back.
X (Twitter)	Official Web / Media Studio Asset Reuse	`SUBMISSION_VERIFIED`	`PUBLIC_VERIFIED` (`2117261340633161838`)	Reused original uploaded media; receipt and 176.4 s duration verified; stored resolution flagged unestablished.
Douyin	CLI Background Browser + HITL	**`UNKNOWN`** (Read-Back Timed Out)	`USER_CONFIRMED_VISIBLE`	**Idempotency Validation:** Timeout latched `UNKNOWN`; blocked automatic retry; operator phone confirmed live work.
Instagram	CLI Background Browser (Reels)	`SUBMISSION_VERIFIED`	`PUBLIC_VERIFIED` (`DfQ9iCiCWYN`)	Single upload and single Share; native receipt, ZH/EN caption, 16:9 ratio, and AI label verified via read-back.

Table 7: Phase-level latency decomposition of the live Instagram publication workflow (*N* = 1 instrumented observation).

Execution Phase	Execution Scope	Duration (s)	% of Cmd	% of E2E
1. Source & Release Plan Cryptographic Checks	Outside Publish Cmd	0.438	—	0.10%
2. Local Dry-Run Rehearsal	Outside Publish Cmd	0.444	—	0.10%
3. Sign-In Preflight & Session Close	Outside Publish Cmd	10.137	—	2.40%
Subtotal: Pre-Publication Governance Checks	***Outside Publish Cmd***	***11.019***	**—**	***2.60%***
4. Browser Start to Media File Entry	Inside `vpub publish`	11.058	2.68%	2.61%
5. Post-Submission Wait (Transfer, Processing & Confirmation)	**Inside `vpub publish`**	**380.551**	**92.32%**	**89.91%**
6. Independent Post-Publication Read-Back	Inside `vpub publish`	5.224	1.27%	1.23%
7. Residual In-Command Form/UI Overhead	Inside `vpub publish`	15.384	3.73%	3.63%
Subtotal: Publication Command Total	***Inside `vpub publish`***	***412.217***	***100.00%***	***97.40%***
Total End-to-End Observed Workflow Time	**All Recorded Phases**	**423.236**	**—**	**100.00%**

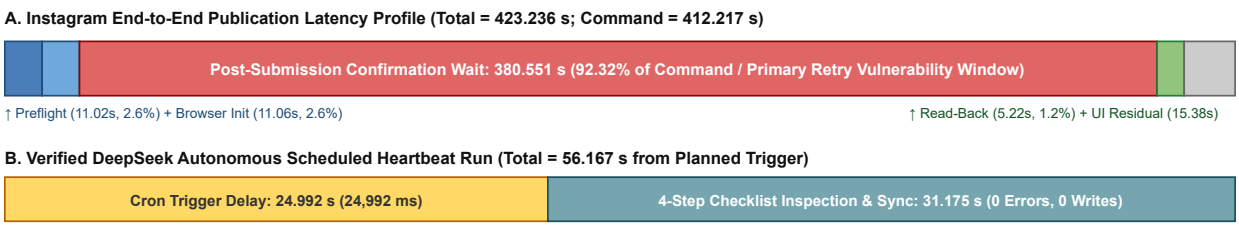


Figure 5. Empirical timeline comparison of (A) the live Instagram browser-mediated publication run (showing the 92.32% post-submission wait window protected by `f_submit = true`) and (B) the verified DeepSeek autonomous heartbeat inspection run.

6.4 Multi-Modal Quality Engineering and Complete 30-Artifact Catalogue (RQ3)

To demonstrate how Layer 2 governs deliverable quality across iterative revisions, Table 8 synthesizes the quantitative frame-level and audio-signal engineering audits conducted across the production studies, and Table 9 lists the complete manifest of all $N = 30$ cryptographically verified video versions archived in *Agentic Work Lab*.

Table 8: Multi-modal programmable video quality engineering cases and surgical defect repair verifications.

Production Case	Diagnosed Technical Defect	Surgical Code / Timeline Repair	Quantitative Verification & Documented Limits
Animation Timing Diagnostic (`animation-timing.md`)	Changing render from 24 to 30 fps failed to create missing contact, load-transfer, and withdrawal actions.	Decomposed 24 fps exposure rules (ones = 24, twos = 12, threes = 8 exp/s) and 5-stage physical handoff criteria.	66.7 s (2,001 frames at 30 fps): 52 segments, 44 holds (median hold 0.85 s, max 7.0 s). Established diagnostic baseline.
GPT Duty Log 05 (`v3-antiflash`, 434.47 s)	Full-opacity frame before fade, white flashes during right-panel changes, and shot leakage at cuts.	Set explicit hidden initial states, frame-aligned boundaries, whole-panel replacement, and 4-frame clean-entry hold; kept v2 audio.	**13,034 / 13,034 frames scanned** at 320×180: 11 sequences fixed; 0 ABA/black/white pulses; -15.97 LUFS, -1.21 dBTP, 0 ms audio offset.
Whale Girl Transformation (`v3` 67s → `v4` 75.23s)	Abrupt eye gaze jumps, brightness mismatch across cut, and moving tail fin entering fixed hair mask.	Interpolated eye regions only between real source endpoints; matched cut brightness; expanded v4 protection mask; ducked music 12 dB.	1080×1920, 2,257 frames: 0 decode errors, 0 clipped samples (-0.636 dBFS peak), all-zero silence verified from 50.763 to 62.503 s, black frame at 1,845 (61.500 s).
Whale Girl Revisions (`r5` Colour/Heels & `r7` Motion)	Colour spill onto blue tail during grading (r5); uneven alternating steps in sock/heart gestures (r7).	Separated clothing/skin/background masks protecting tail & 6 heel events (r5); reordered real source frames without optical flow (r7).	r5: 66.87 s (2,006 frames), full decode passed (+1.11 dBFS inherited float peak noted). r7: 65.37 s (1,961 frames), join & duplicate-frame checks passed.
Whale Girl Ep 07 & Portrait Cuts A/B/C (179.77 s + 3 cuts)	Cropping 1920×1080 landscape to 1080×1920 portrait cuts off ticket counters and character art.	Programmatically rearranged ticket/character panels into $y \in [168, 1330]$ (56 px subtitles, 106 px numbers) at 360 px preview width.	5,393 landscape frames + 843/897/899 portrait frames; -17.5/-16.6/-17.5 LUFS, -0.5 dBFS peak, 0 clips. Documented 130 ms first-ticket stamp offset vs. earlier 6 ms estimate.

Table 9: Complete cryptographic manifest and verification status of all $N = 30$ downloadable video delivery versions archived in *Agentic Work Lab* (2,211,716,129 bytes / 2.21 GB total; 100% SHA-256 verified).

#	Archive Filename	Deliverable Title / Scope	MB	Exact Bytes	SHA-256 Prefix	Dur (s)	State
1	`gpt-log-01.mp4`	GPT Duty Log 01	153.01	160,441,478	`13a9b82396e5da…`	—	`published`
2	`gpt-log-02.mp4`	GPT Duty Log 02	164.55	172,545,563	`decb95dbb5ebaa…`	—	`published`
3	`gpt-log-03.mp4`	GPT Duty Log 03	42.38	44,443,753	`b6932ebc4f5821…`	—	`published`
4	`whale-log-01.mp4`	Whale Girl Shift Log 01	40.21	42,158,416	`5fc2a8bc961a49…`	—	`published`

#	Archive Filename	Deliverable Title / Scope	MB	Exact Bytes	SHA-256 Prefix	Dur (s)	State
5	`whale-log-02.mp4`	Whale Girl Shift Log 02	33.70	35,336,986	`6bd55d5a62026f…`	—	`published`
6	`whale-log-03.mp4`	Whale Girl Shift Log 03	23.84	24,994,028	`95a862fd0e9ed8…`	—	`published`
7	`gpt-log-04.mp4`	GPT Duty Log 04: production tutorial	157.70	165,360,386	`870fd0dd15459a…`	—	`published`
8	`gpt-log-05-review.mp4`	GPT Episode 05: Seedance beginner tutorial (review copy)	105.48	110,600,280	`2e4a5da549b341…`	—	`ready_for_review`
9	`archive-episode03-candidate.mp4`	Earlier local episode 03 candidate (not the released episode)	169.51	177,747,769	`3c2225790f5f17…`	—	`unpublished_experiment`
10	`archive-episode04-story-v4.mp4`	Earlier local episode 04 story cut (not the released episode)	67.51	70,793,338	`96da49a29f44ae…`	—	`unpublished_experiment`
11	`archive-triangle-cats.mp4`	Triangle-cats challenge candidate	72.21	75,712,985	`7c081526da7a24…`	—	`unpublished_experiment`
12	`archive-dinner-full-film.mp4`	Earlier complete dinner-animation experiment	130.34	136,670,959	`6b1927cc279e2e…`	—	`unpublished_experiment`
13	`study-connected-motion-v6.mp4`	Ten-second continuity experiment v6	2.44	2,562,181	`41a2929321d5f3…`	—	`research_sample`
14	`study-comic-v3-1.mp4`	Comic-style short experiment v3.1	13.78	14,449,925	`adef504edb059f…`	—	`research_sample`
15	`study-3d-handover-v2.mp4`	3D walk-and-handoff prototype v2	23.56	24,704,815	`b409355012deff…`	—	`research_sample`
16	`study-seedance-previs.mp4`	Seedance production previs	6.20	6,500,102	`33132eddbeb8a1…`	—	`research_sample`
17	`whale-log-04-v2-20261028.mp4`	Whale Girl Shift Log 04: small favours	11.93	12,506,936	`c23c171149bc47…`	132.00	`scheduled_on_bilibili`
18	`whale-log-05-v2-20261028.mp4`	Whale Girl Shift Log 05: the bill	13.40	14,052,172	`f735aca1bb770a…`	148.20	`scheduled_on_bilibili`
19	`gpt-log-05-v3-antiflash-20261028.mp4`	GPT Duty Log 05: Seedance tutorial (flash correction)	83.52	87,578,698	`5dc121e833dcfc…`	434.47	`scheduled_on_bilibili`

#	Archive Filename	Deliverable Title / Scope	MB	Exact Bytes	SHA-256 Prefix	Dur (s)	State
20	`gpt-log-06-v2-20261029.mp4`	GPT Duty Log 06: Step by step, getting DeepSeek to help with my work (v2)	42.23	44,282,436	`b53d3df6f85b2e…`	591.47	`scheduled_on_bilibili`
21	`whale-log-06-v2-20261029.mp4`	Whale Girl Shift Log 06: AI does the automation; I pretend to be human (v2)	13.88	14,552,237	`35bc463bc4c7fc…`	184.20	`scheduled_on_bilibili`
22	`study-whale-transformation-v3-20261030.mp4`	Whale Girl transformation study v3 (earlier delivery)	85.84	90,012,465	`60f03a12439c0c…`	67.00	`unpublished_experiment`
23	`study-whale-transformation-v4-20261030.mp4`	Whale Girl transformation study v4 (targeted repair)	98.33	103,106,107	`3b209af916b128…`	75.23	`unpublished_experiment`
24	`study-whale-color-heels-r5-20261031.mp4`	Whale Girl study: local colour and heel-sound revision 5	108.10	113,352,299	`19ea894908ce36…`	66.87	`not_verified`
25	`study-whale-motion-r7-20261101.mp4`	Whale Girl study: motion-timing revision 7	103.48	108,503,225	`933e1643a73519…`	65.37	`not_verified`
26	`whale-log-07-v3-20261101.mp4`	Whale Girl Shift Log 07: duty simulator (v3)	47.02	49,307,630	`e37285ee4f8c9f…`	179.77	`not_refreshed`
27	`whale-log-07-cut-a-v3-20261101.mp4`	Episode 07 portrait cut A: running out of resources	7.50	7,861,825	`8429157c4fe444…`	28.10	`not_refreshed`
28	`whale-log-07-cut-b-v3-20261101.mp4`	Episode 07 portrait cut B: refusing every request	7.81	8,185,740	`1e436aff188953…`	29.90	`not_refreshed`
29	`whale-log-07-cut-c-v3-20261101.mp4`	Episode 07 portrait cut C: bluffing through the shift	7.19	7,536,390	`e7ba91f7e903e2…`	29.97	`not_refreshed`
30	`i-watched-it-for-you-v3-20261102.mp4`	I Watched It For You (v3)	272.61	285,855,005	`7d6fd76090f7a9…`	176.40	`not_refreshed`
30	**Total Verified Archive**	**7 GitHub Release Snapshots (0 Mismatches)**	**2,109.26**	**2,211,716,129**	**`100% Matched`**	—	**`Verified`**

6.5 Descriptive Cohort Analytics and 12-Scenario Governance Ablation (RQ3)

Table 10 reports the descriptive statistical distributions across all $N = 55$ channel records (comparing the $n = 45$ historical human-led cohort against the $n = 10$ AI-assisted series cohort and 24-hour checkpoints), followed by Table 11, which evaluates the proposed governance architecture against two baseline automation paradigms across 12 real operational fault modes encountered during the project.

Table 10: Descriptive statistical comparison of the historical human-led cohort ($n = 45$ published) and the AI-assisted series cohort ($n = 10$ total: 5 published, 5 scheduled) at the 28 October 2026 snapshot, alongside 24-hour checkpoints.

Cohort / Metric	Mean	SD	Median	IQR (Q1–Q3)	Min–Max	Sum
A. Historical Human-Led Cohort (human_led_history: n = 45 published, Oct 2019 – Oct 2026)						
Lifetime Views	40,884.31	64,604.81	15,407.0	1,722–67,502	274–333,725	1,839,794
Likes	4,022.36	7,281.57	729.0	89–3,765	8–30,299	181,006
Comments	78.18	132.65	14.0	4–113	0–597	3,518
Favorites	1,584.31	2,048.27	473.0	19–2,746	2–6,158	71,294
Coins	356.38	751.32	54.0	6–237	2–3,044	16,037
Shares	764.76	1,594.05	31.0	1–996	0–5,571	34,414
Danmaku (Bullet Chats)	190.27	232.37	105.0	41–279	0–1,045	8,562
B. Current AI-Assisted Series (current_ai_series: n = 5 published, 5 scheduled, 25–28 Oct 2026)						
Early Views (≤ 3 days age)	133.20	99.53	183.0	42–199	11–231	666
Likes	10.60	7.70	9.0	8–11	2–23	53
Comments	29.40	46.94	12.0	7–13	2–113	147
Favorites	4.20	5.26	2.0	1–5	0–13	21
C. Age-Matched 24-Hour Post-Publication Checkpoints (Single-Work Descriptive Records)						
GPT Duty Log 03 (`BV1UXxN6aECf` , 70 s, age 24.38 h): 229 views, 8 likes (3.49%), 7 replies (3.06%), 44.64% completion (31 s avg watch).						
Whale Girl 02 (`BV1Wfx56JEih` , 117 s, age 24.05 h): 308 views, 24 likes (7.79%), 51 replies (16.56%), 36.63% completion (42 s avg watch).						

As shown in Table 10, the historical human-led cohort ($n = 45$ works published between October 2019 and October 2026) exhibits a highly right-skewed lifetime view distribution (Mean = 40,884.31, SD = 64,604.81, Median = 15,407.0, IQR = 1,722–67,502, cumulative sum = 1,839,794 views). By contrast, the newly launched AI-assisted series cohort at the 28 October 2026 snapshot ($n = 5$ published works with exposure ages ≤ 3 days, plus 5 scheduled works) recorded a mean of 133.20 early views (Median = 183.0). In an ungoverned reporting workflow, comparing these two raw means would either falsely suggest that AI-assisted production underperforms by two orders of magnitude (ignoring 7 years of cumulative platform exposure) or, conversely, allow cherry-picked channel traffic spikes (such as a 76,000-play weekly channel fluctuation driven by historical back-catalog videos) to be misattributed to the new AI series. By enforcing age-matched 24-hour checkpoints (Panel C: 44.64% completion on *GPT Duty Log 03* at 24.38 h and 36.63% completion on *Whale Girl 02* at 24.05 h) and machine-readable `descriptive_snapshot_not_causal` schema constraints, *Agentic Work Lab* prevents confounded causal claims. Finally, Table 11 synthesizes a trace-driven ablation study comparing our four-layer architecture against two baselines across 12 empirical fault scenarios.

Table 11: Trace-driven governance ablation across 12 empirical fault and exception scenarios recorded in the laboratory.

Empirical Fault / Exception Scenario	Baseline A: Ungoverned Agent Retry Loop	Baseline B: Schedule Gate Only (No Receipt Latch)	Proposed 4-Layer Governance (Lab + CrossCast)
1. Douyin post-submit read-back timeout (Nov 3 film)	**FAIL:** Re-uploads 285.8 MB video, creating duplicate public post.	**FAIL:** Retries submission after timeout expires.	**PASS:** Latches `UNKNOWN` (`f_submit=true`); blocks retry; verified via HITL phone check.
2. Instagram 380.55 s confirmation wait window	**FAIL:** Agent timeout triggers second Share click.	**WARN:** Vulnerable if CLI session disconnects mid-wait.	**PASS:** Single Share persisted before wait; explicit wait progress (#13) & read-back verification.
3. Non-owner / old agent executor wake-up	**FAIL:** Executes duplicate checklist actions concurrently.	**PASS:** Blocked by owner check.	**PASS:** Deterministic `planRun` returns `(skip, not_owner)` + OS `process_lock.py`.
4. Hourly heartbeat during active render / QA pending	**FAIL:** Publishes unverified intermediate render (e.g., GPT Ep 06).	**FAIL:** Publishes if due time passes without QA gate.	**PASS:** Excluded via `authorized != true` and `RENDER_OR_QA_PENDING` gate.
5. Unpatched flash frames in tutorial render (GPT Ep 05)	**FAIL:** Releases defective video with white/black pulses.	**FAIL:** Hash check passes uninspected first render.	**PASS:** 13,034-frame scan detects 11 flash sequences; surgical repair retains v2 audio master.
6. Missing native Edit button on published X post	**FAIL:** Deletes original post and re-uploads (losing replies/ID).	**FAIL:** Aborts without state reconciliation.	**PASS:** Strictly forbids delete/repost fallback; preserves original post & logs policy update.
7. YouTube Studio Save button re-arming latency	**FAIL:** Double-clicks Save, corrupting draft state.	**WARN:** Unverified save state.	**PASS:** Verifies after one final Save without re-arming button (PR #16, `0969064`).
8. Imprecise pinned-comment selector (`988bee2` trial)	**FAIL:** Pins wrong comment on live creator page.	**FAIL:** Lacks counterexample gate before live action.	**PASS:** Independent counterexample test caught selector flaw; live pin held until fixed.
9. X scheduled post row lacking durable media/post ID	**FAIL:** Marks schedule as fully verified public post.	**WARN:** Over-claims verification status.	**PASS:** Retains `SUBMITTED_UNVERIFIED` receipt; cleans up test schedule & confirms absence.
10. Single-work deletion of test upload (IG / X / YT)	**FAIL:** Risk of deleting adjacent production works.	**FAIL:** No post-delete absence verification.	**PASS:** Durable consent token + exact identity match + refreshed absence read-back (PR #29/#30).
11. Browser worker interrupted mid-login or cleanup	**FAIL:** Leaves zombie browser lock blocking future runs.	**FAIL:** Stale lock requires manual file deletion.	**PASS:** Confirms OS PID exit before lock recovery; isolates cleanup from publish path (#27/#28).
12. Confounded lifetime view comparison (45 vs. 5 works)	**FAIL:** Falsely claims AI series underperforms or drives 76k weekly plays.	**FAIL:** Reports raw means without exposure-age control.	**PASS:** Automatically embeds `descriptive_snapshot_not_causal` metadata & 24h age buckets.

7. Discussion

7.1 Theoretical Implications for Project Management and Human–AI Governance

Our findings contribute to the emerging theory of AI-augmented project management (Holzmann et al., 2022; Müller et al., 2024; Raisch & Krakowski, 2021) in three fundamental ways. First, whereas prior multi-agent frameworks (Wu et al., 2024; Hong et al., 2024; Qian et al., 2024) rely primarily on natural-language role prompts to coordinate task handoffs, our empirical trace shows that safe delegation in continuous operations requires externalizing stage-gate controls into deterministic code invariants (`planRun` , OS process locks, and `--confirm CURRENT_PLAN_HASH`). This bridges classical project configuration management (Whyte et al., 2016; Cooper, 1990) with modern agentic systems engineering. Second, in human–automation interaction theory (Parasuraman et al., 2000; Shneiderman, 2020), automation authority must be *state-dependent*: agents can operate with high autonomy during preflight verification and byte staging (States 1–3), but must immediately surrender retry authority to a conservative latch (`UNKNOWN_LATCHED` , State 5a) once an external commit flag is set. Third, by embedding provenance tracking, SHA-256 artifact manifests, and non-causal limitation tags directly into the reporting pipeline, *Agentic Work Lab* establishes epistemic governance as a first-class project management deliverable.

7.2 Managerial and Engineering Lessons Learned

- **Measure the Wait Before Optimizing Concurrency:** Because 92.32% of live browser-mediated publication time occurs inside platform confirmation waits after submission (Table 7), optimizing local script startup yields negligible end-to-end speedup; engineering effort should focus on safe session isolation and bounded two-slot cross-platform concurrency (Figure 4).
- **Design Editable Intermediate Representations:** Decomposing deliverables into code-driven timelines, separate audio tracks, and structured JSON patches allows human project managers to enforce surgical revisions (Table 8) without discarding validated upstream work.
- **Preserve Original State on Verification Interruptions:** When platform security challenges or missing edit permissions occur, tools must never automatically delete, repost, or overwrite existing works.

7.3 Threats to Validity and Limitations

Consistent with software engineering case study guidelines (Runeson & Höst, 2009), we explicitly document the boundaries of this study: (1) *Internal Validity:* The Instagram latency decomposition (412.217 s) and the verified DeepSeek scheduled run (31.175 s) represent exact instrumented single-run observations (N = 1 each) rather than multi-trial statistical benchmarks, and client-side measurements cannot separate upload transfer from server-side transcoding during the 380.551 s confirmation wait. Furthermore, an intermittent local file-read hash discrepancy observed once prior to archiving *I Watched It For You v3* was resolved on subsequent reads, but its underlying OS/storage root cause was not over-claimed as repaired. (2) *Construct Validity:* The pre-registered blinded identical-task multi-model merchant evaluation (comparing GPT and DeepSeek on identical prompts with token-cost logging) remains an unexecuted plan and is not claimed as an empirical result here. (3) *External Validity:* Empirical records derive from a single creator/engineering laboratory across five platforms (with TikTok integration suspended due to regional authentication constraints).

8. Conclusion and Future Work

This paper presented a Design Science study addressing how to govern human–AI collaborative project workflows in multi-platform digital delivery. By architecting and evaluating *Agentic Work Lab* and *PublishOps CrossCast*, we demonstrated that reliable multi-agent operations require combining clear role handoff boundaries (across human project management, GPT/Codex, Claude, and DeepSeek) with a four-layer deterministic governance framework: (1) sole-operator and temporal scheduling gates, (2) cryptographic plan-hash and multi-modal media verification, (3) an idempotent receipt state machine that latches `UNKNOWN` post-submission states to eliminate duplicate retries, and (4) independent read-back, identity-bound in-place revision, and human-in-the-loop verification handoffs. Empirical evidence across 69 operational events, 30 SHA-256 verified video versions (2.21 GB), 2,747 automated tests, and five live publishing platforms confirms that the architecture prevents unauthorized writes and duplicate publications while maintaining a transparent, reproducible audit trail.

Data and Code Availability Statement

All source code, workflow verification scripts, de-identified operational event logs (N = 69), channel cohort snapshots (N = 55), and SHA-256 verified media release manifests (N = 30) are openly available at `https://github.com/whitecarp/agentic-work-lab` and `https://github.com/whitecarp/publishops-crosscast`.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Adams, B., & McIntosh, S. (2016). Modern release engineering in a nutshell—Why researchers should care. *IEEE SANER*, 5, 78–90.

2. Amershi, S., Weld, D., Vorvoreanu, M., et al. (2019). Guidelines for human-AI interaction. *Proceedings of ACM CHI*, 1–13.
3. Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional.
4. Berente, N., Gu, B., Recker, J., & Santhanam, R. (2021). Managing artificial intelligence. *MIS Quarterly*, 45(3), 1433–1450.
5. Bianchi, M. J., Conforto, E. C., Rebentisch, E., et al. (2022). Stage-Gate and agile project management: A framework for hybrid models. *IEEE Transactions on Engineering Management*, 69(6), 3590–3603.
6. Cooper, R. G. (1990). Stage-Gate systems: A new tool for managing new products. *Business Horizons*, 33(3), 44–54.
7. Flyvbjerg, B. (2006). Five misunderstandings about case-study research. *Qualitative Inquiry*, 12(2), 219–245.
8. Gray, J., & Reuter, A. (1992). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.
9. Gregor, S., & Hevner, A. R. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37(2), 337–355.
10. Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.
11. Holzmann, V., Zitter, D., & Peshkess, S. (2022). The expectations of project managers from artificial intelligence: A Delphi study. *Project Management Journal*, 53(5), 438–455.
12. Hong, S., Zhuge, M., Chen, J., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. *ICLR*.
13. Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
14. Kerzner, H. (2022). *Project Management: A Systems Approach to Planning, Scheduling, and Controlling* (13th ed.). Wiley.
15. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
16. Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565.
17. Leite, L., Rocha, C., Kon, F., Milojicic, D., & Meirelles, P. (2019). A survey of DevOps concepts and challenges. *ACM Computing Surveys*, 52(6), 1–35.
18. Müller, R., Locatelli, G., & Holzmann, V. (2024). Artificial intelligence and project management: Governance, agency, and human-AI collaboration. *International Journal of Project Management*, 42(1), 102554.
19. Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A*, 30(3), 286–297.
20. Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77.
21. Project Management Institute (PMI). (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* (7th ed.). PMI.
22. Qian, C., Liu, W., Liu, H., et al. (2024). ChatDev: Communicative agents for software development. *Proceedings of ACL*, 15174–15186.
23. Raisch, S., & Krakowski, S. (2021). Artificial intelligence and management: The automation–augmentation paradox. *Academy of Management Review*, 46(1), 192–210.
24. Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164.
25. Schneider, F. B. (1990). Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4), 299–319.
26. Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review. *IEEE Access*, 5, 3909–3943.
27. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 36, 8634–8652.
28. Shneiderman, B. (2020). Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction*, 36(6), 495–504.
29. Turner, J. R. (2014). *Handbook of Project-Based Management* (4th ed.). McGraw-Hill Education.
30. Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems*, 25(1), 77–89.
31. Wang, L., Ma, C., Feng, X., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345.
32. Whyte, J., Stasis, A., & Lindkvist, C. (2016). Managing change in the delivery of complex projects: Configuration management, asset information and 'big data'. *International Journal of Project Management*, 34(2), 339–351.
33. Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). John Wiley & Sons.
34. Wu, Q., Bansal, G., Zhang, J., et al. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *COLM*.
35. Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR*.